List Functions
Recursion w Lists

CSC345: Programming Languages and Paradigms

WCU
WEST CHESTER
UNIVERSITY

Today

1. List Functions
2. Recursion with Lists

Useful List Library Functions to know

Defined already in the “standard prelude”:

```
head [1,2,3,4,5]
```
Select the first element of a nonempty list

```
tail [1,2,3,4,5]
```
Return everything but the first element of a nonempty list

```
[1,2,3,4,5] !! 2
```
Select the nth element of a list (zero-indexed)

```
take 3 [1,2,3,4,5]
```
Select the first n elements of a list

```
drop 3 [1,2,3,4,5]
```
Return everything but the first n elements of a list

```
length [1,2,3,4,5]
```
Return the length of a list

```
sum [1,2,3,4,5]
```

Calculate the sum of a list of numbers

```
product [1,2,3,4,5]
```

Calculate the product of a list of numbers

```
reverse [1,2,3,4,5]
```

Return the list, reversed

```
[1,2,3] ++ [4,5]
```

Append two lists

```
1 : [2,3,4,5]
```

Construct a new list by prepending an element to the start of a list

```
null []
```

Returns whether or not the list is empty

```
and [True, False, True]
```
Return the conjunction of a list of booleans

```
or [True, False, True]
```
Return the disjunction of a list of booleans

```
last [1,2,3,4,5]
```
Return the last element of a list

```
init [1,2,3,4,5]
```
Return all but the last element of a list

```
replicate 3 'c'
```
Create a list of n copies of an element

```
splitAt 3 [1,2,3,4,5]
```
Split a list at a given position

zip

- Produces a new list by pairing successive elements from two lists until either or both lists are exhausted

```
> zip [‘a’,’b’,’c’] [1,2,3,4]
[(‘a’, 1), (‘b’, 2), (‘c’, 3)]
```

unzip

- Takes a list of pairs
- Produces a pair of lists, where each list is composed of elements at the same position in every pair

```
> unzip [('a', 1), ('b', 2), ('c', 3)]
(['a','b','c'] [1,2,3])
```

Example 1

How can we define `length` on our own, using what we already know?

length’ :: [Int] -> [Int]

Example 2

Write a function `lowers` that returns how many lowercase letters there are in a string.

```
lowers :: String -> Int
```

Example 3

Write a function `pairs` that returns the list of all pairs of adjacent elements from a list.

```
pairs :: [Int] -> [(Int, Int)]
```

Example usage:

```
> pairs [1,2,3,4]
[(1,2), (2,3), (3,4)]
```

Example 4

Write a function `sorted` that returns if a list of elements is sorted.

```
sorted :: [Int] -> Bool
```

Recursion on Lists

Need:

1. Base case
2. Recursive step

Common setup:

- Base case is empty list
- Recursive step operates on nonempty list

Pattern Matching on Lists

- *Functional programming style:* use pattern matching to check whether a list is empty or non-empty
  - Can bind identifiers to parts of the non-empty list

Rules:

1. A list matches pattern [] when it is empty
2. A list matches pattern (p:ps) if it is non-empty, and also if its head matches the pattern p and its tail matches the pattern ps

Pattern Matching Examples

List	Pattern	What happens
[2]	(p:ps)	• 2 matched with p • [] matched with ps
[2,3,4]	(p:ps)	• 2 matched with p • [3,4] matched with ps
[2,3,4]	(q:p:ps)	• 2 matched with q • 3 matched with p • [4] matched with ps
[2,3,4]	(q:3:ps)	• 2 matched with q • 3 matched with 3 • [4] matched with ps
[2]	(q:p:ps)	• 2 matched with q • [] matched with p • ps *does not match* • Overall, pattern does not match

Example 1

Defining list library functions for ourselves using recursion:

How can we define `sum` on our own, using recursion?

```
sum' :: [Int] -> Int
```

Example 2a and 2b

```
product’ :: [Int] -> Int
```

```
length’ :: [Int] -> Int
```

Example 3

```
reverse' :: [Int] -> [Int]
```

Example 4

How can we define ++ (append) on our own, using recursion?

```
(+++) :: [Int] -> [Int] -> [Int]
```

Comparing Recursion w/ List Comprehension

```
> doubleAll [2,3]
[4,6]
```

Comparing Recursion w/ List Comprehension

```
> selectEven [2,3]
[2]
```

Insertion Sort

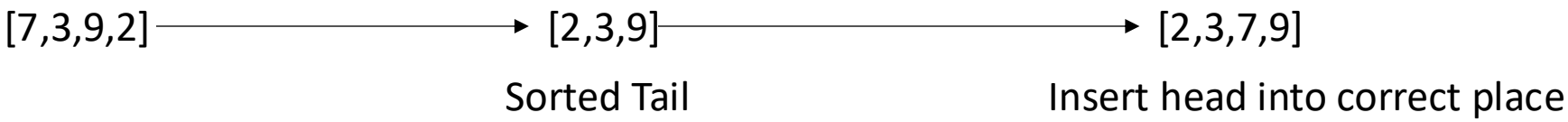
Two functions to write:

1. Insert the item into a given sorted list

   ins :: Int -> [Int] -> [Int]

2. Insertion sort
   - *Idea:* Sort the tail of the list, and then insert the head into the correct place

   iSort :: [Int] -> [Int]



Insertion Sort (1)

```
ins :: Int -> [Int] -> [Int]
```

Insertion Sort (2)

```
iSort :: [Int] -> [Int]
```

Example Trace

iSort [7,3,9,2]